



tecnun
Universidad
de Navarra

TELEMATICS SYSTEMS

Emulation of a distributed control system of a car

May, 2026

Paul Auzmendi
Ander Melendez

INDEX

INTRODUCTION	3
OVERVIEW OF THE PROJECT	4
MASTER	6
1. Classes	6
2. States of the Master	9
3. Flowcharts of the Main Functions	10
SLAVE 1: MOTOR	15
1. Classes	15
2. States of the Slave 1: Motor	17
3. Flowchart of the Slave 1: Motor	18
SLAVE 2: ACCIONAMIENTOS	20
1. Classes	20
2. States of the Slave 2: Accionamientos	22
3. Flowchart of the Slave 2: Accionamientos	22
SLAVE 3: LUCES	24
1. Classes	24
2. States of the Slave 3: Luces	26
3. Flowchart of the Slave 3: Luces	27
WEB SERVER	29
CONCLUSION	31

INTRODUCTION

This report describes the design and implementation of the final project for the Telematic Systems course: a distributed control system that emulates the electronic architecture of a car. The system is built around four independent Windows applications developed in C++ with the MFC framework, communicating over TCP/IP using the Modbus protocol.

The architecture follows a master-slave model. A central application, referred to as the “Centralita”, acts as the Modbus master and polls three slave devices that simulate different subsystems of the vehicle: the engine (temperature and RPM sensors), the actuators (brake and turn signals), and the lights (front and rear indicators, brake light). The master periodically reads data from the engine and actuator slaves and writes commands to the lights slave, reproducing the typical request-response behavior of a Modbus network.

In addition to the desktop interface, the Centralita embeds a lightweight HTTP server that exposes the current state of the vehicle through a web page. This page can be opened in any browser and displays the engine parameters and active lights in real time, providing a remote visualization layer on top of the Modbus network.

The project also makes use of concurrent programming. The Centralita runs several worker threads in parallel, one per slave, so that the polling loop never blocks the user interface and the three subsystems can be queried simultaneously.

The remainder of this document is organized as follows. Section 2 gives a general overview of the system and its communication flow. Sections 3 to 6 describe each application individually (the Centralita and the three slaves) including their user interface, internal structure, and main classes. Section 7 covers the embedded web server. Finally, the report ends with a brief conclusion summarizing the work.

OVERVIEW OF THE PROJECT

The system is built as a small distributed network of four independent Windows applications that exchange information over TCP/IP. One of them, the Centralita, acts as the central node and concentrates all the intelligence: it polls the other three applications (the slaves), keeps an internal model of the vehicle state, and exposes that state to the outside world through an embedded web server.

The communication between the Centralita and the slaves follows the Modbus TCP protocol. Each slave listens on its own TCP port and replies to Modbus requests sent by the master. Two function codes are used in the project: 0x03 (Read Holding Registers) to read sensor values and switch states from the Motor and Accionamientos slaves, and 0x06 (Write Single Register) to send commands to the Luces slave.

The role of each application is the following:

Centralita (Master + Web Server). Connects to the three slaves through three independent worker threads. It polls the engine values and the actuator switches at a configurable interval and forwards the actuator state to the Luces slave as Modbus write commands. In parallel, it runs a small HTTP server that serves a dashboard page and a JSON endpoint, so the whole vehicle state can be visualized in any web browser.

Slave 1: Motor. Simulates two engine sensors, temperature (0–300 °C) and RPM (0–7000), driven by two sliders. It listens on port 3502 and answers read requests for the registers 400 and 401.

Slave 2: Accionamientos. Simulates the driver inputs (brake, left turn signal and right turn signal) implemented as three check buttons. It listens on port 3503 and answers read requests for the registers 400, 401 and 402.

Slave Luces 3. Simulates the physical lights of the vehicle: brake light, two front turn signals and two rear turn signals. It listens on port 3504 and reacts to write commands on the registers 500 to 504, switching the corresponding LEDs on and off.

The diagram below (Figure 1) summarizes the architecture and the direction of the messages exchanged between the components.

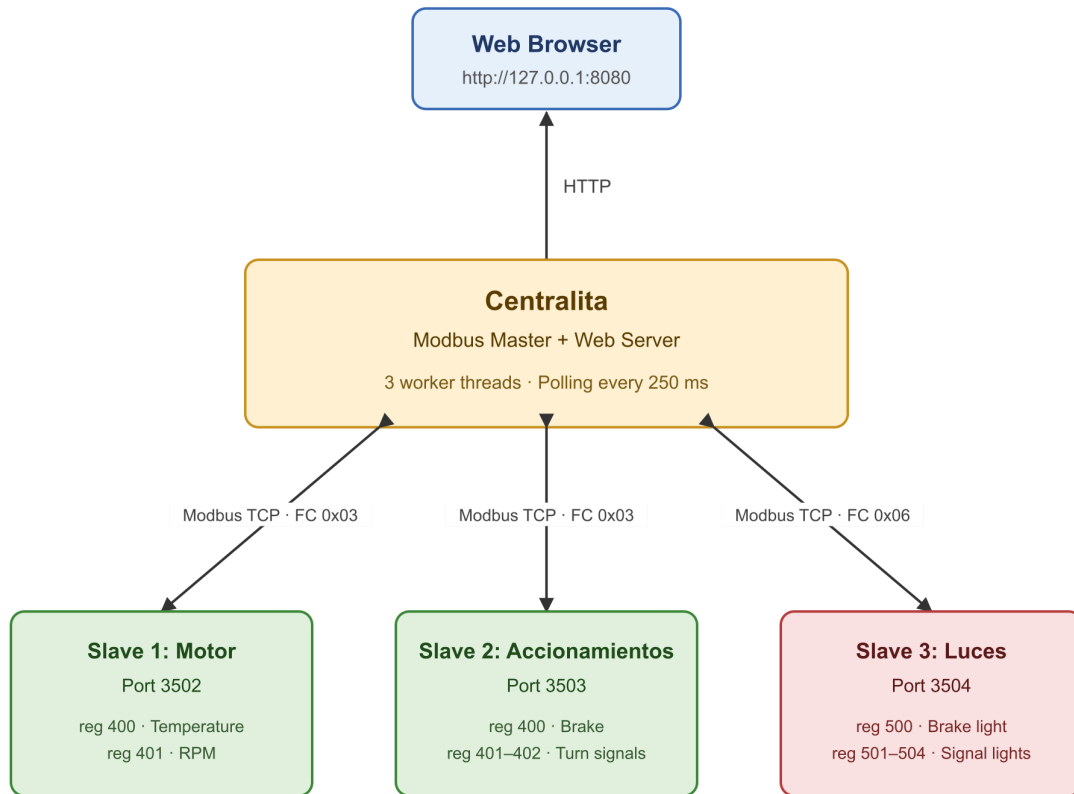


Figure 1 – General overview of the project

MASTER

This section describes the classes, functions and states of the “Centralita”, which works as the master in communications and launches the web server. Figure 2 shows the UI of the Master application.

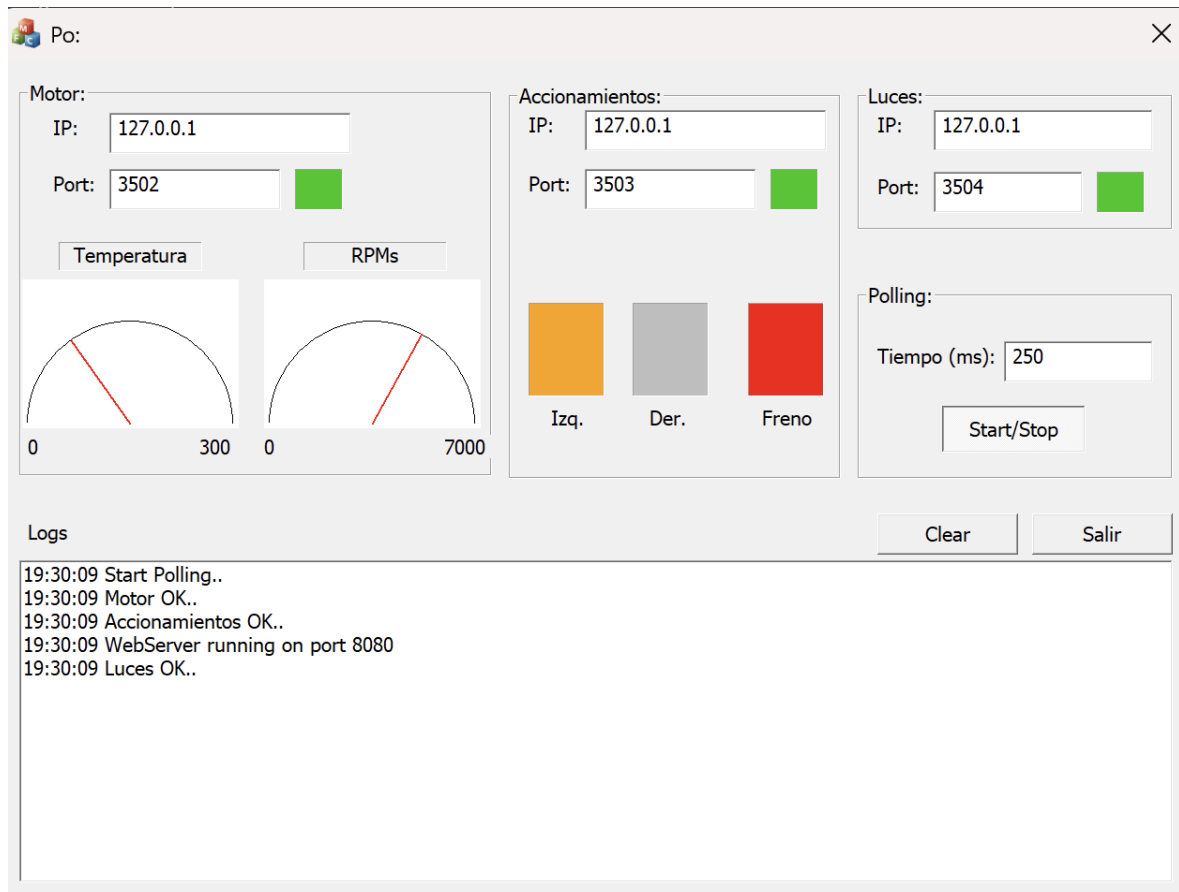


Figure 2 – Centralita

1. Classes

The Centralita is built around four classes. Three of them derive from the MFC framework and follow the standard MFC patterns; the fourth (Modbus) is a small utility class that is shared, unchanged, with the three slave applications.

CMaestroDlg. Derived from CDialogEx.

This is the main dialog class. It owns the user interface (gauges, LEDs, log list and configuration fields), holds the connection parameters for the three slaves, and stores the shared state that the worker threads read and write (start, m_tempValue, m_rpmValue, m_frenoState, m_izqState, m_derState). It also owns the listening socket of the embedded web server. Its most relevant methods are:

- **OnInitDialog()** — initializes Winsock with `AfxSocketInit`, sets the colors of the on-screen LEDs and starts the timer that drives the blinking of the turn signals.
- **OnClickedStart()** — when the Start button is pressed, launches the three worker thread functions (Motor, Accionamientos, Luces) with `AfxBeginThread` and starts the web server by calling `Create` and `Listen` on `s_listen`.
- **OnTimer()** — fires every 400 ms and calls `Tick()` on the turn-signal LEDs to animate their blinking.
- **OnLogMsg(WPARAM, LPARAM)** — message handler registered for the user-defined message `WM_USER_LOG`. The worker threads post log lines as `CString*` pointers through `PostMessage`, and this handler appends them to the list box. This allows the 3 threads to be able to write in the same `ListBox` without crashing the application.
- **OnWebAccept()** — called by `CWebSocket` whenever a browser opens a connection. Reads the HTTP request, decides whether to serve the HTML dashboard or the JSON state endpoint, and writes the response.

CWebSocket: Derived from CAsyncSocket.

In MFC, the Windows Sockets API is wrapped by two classes: `CAsyncSocket` (non-blocking, event-driven) and `CSocket` (blocking). The standard pattern to manage the events of a socket is to derive a new class from one of them and override the virtual methods that correspond to the events of interest (typically `OnAccept`, `OnReceive`, `OnClose` or `OnConnect`). MFC then calls these methods automatically whenever the corresponding event happens on the socket.

`CWebSocket` follows this pattern. It is derived from `CAsyncSocket` and overrides a single virtual method, **OnAccept**, which is automatically executed every time a client (a web browser, in this case) tries to open a connection to the listening socket of the embedded server. The override does not contain any application logic itself; it simply forwards the event to the main dialog, where the HTTP request is read and the response is generated:

Led: Derived from CStatic.

A custom control that replaces the picture controls used elsewhere on the dialog to draw the on-screen indicators. The class wraps three pieces of state: the colour used when the LED is on (`m_color`), the current operating mode (`m_mode`), and a flag (`m_blinkVisible`) that toggles on every timer tick to produce the blinking animation.

The control supports three modes, declared as a public enum: `OFF`, `ON_SOLID` and `ON_BLINKING`. **SetMode** switches between them and triggers a repaint. The painting itself is entirely contained in **OnPaint**. **SetColor(COLORREF)** assigns the color used when the LED is on.

The blinking effect itself is driven externally by the dialog's timer. Every 400 ms the dialog calls **Tick()** on each LED. `Tick` always toggles the internal `m_blinkVisible` flag, but only invalidates the window (forcing a repaint) when the LED is in `ON_BLINKING` mode. This

keeps the cost of an LED in OFF or ON_SOLID mode essentially zero, even though the timer keeps firing in the background.

Modbus.

Utility class that encapsulates the construction and parsing of Modbus TCP frames. The same class is reused unchanged in the three slave applications, which is why it exposes methods for both sides of the conversation. Internally it acts as a small container: the caller sets a few public fields, calls a Build method to obtain the raw bytes of the request, and then later calls a Parse method to decode the response back into those same fields.

The state of the class mirrors the layout of a Modbus TCP frame. The MBAP header is represented by four fields: transactionId, protocolId, length and unitId; and the PDU is represented by functionCode, address, and either value (for single-register operations) or quantity (for read requests). A 125-element array registers[] is provided to hold the values returned by a multi-register read.

On the master side, **BuildReadRequest** and **BuildWriteRequest** produce twelve-byte buffers ready to be sent over the socket. Both methods write the MBAP header (transaction ID, protocol ID = 0, length = 6, unit ID), followed by the function code (0x03 or 0x06), the register address and the quantity or value. All multi-byte fields are packed in big-endian order (network byte order), as required by the Modbus TCP specification.

ParseResponse does the inverse work. It first extracts the MBAP header and the function code, and then dispatches on the latter. For a 0x03 response it reads the byte count and stores the returned register values into registers[], with the first one also copied to value as a shortcut for single-register reads. For a 0x06 response it reads back the echoed address and value, which the master uses to confirm that the write was acknowledged by the slave. The slave-side methods (**ParseRequest**, **BuildReadResponse**, **BuildWriteResponseEcho**) follow the same pattern but in the opposite direction, and are described in the sections of the corresponding slaves.

Figure 3 contains all these classes.

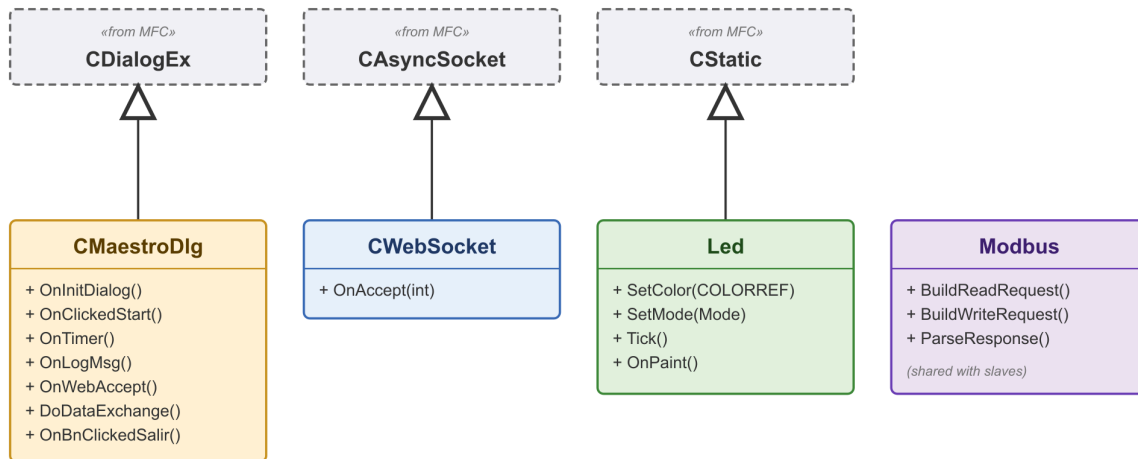


Figure 3 – Classes used in the Master

2. States of the Master

The Centralita has a clear life cycle that can be described with a state diagram (Figure 4). At the top level the application has two states — Idle and Polling Active — and the transitions between them are driven entirely by the Start button (implemented as a push-like checkbox that flips the global flag start).

When the application is launched it sits in the Idle state. No worker threads are running, the embedded web server is not accepting connections, and the connection LEDs are all gray. The user can edit the configuration fields (IP addresses, ports, polling interval) freely in this state.

Pressing Start moves the application into the Polling Active state. Three things happen on entry: the three worker threads (Motor, Accionamientos, Luces) are launched with `AfxBeginThread`, the web server's listening socket is created and put into listen mode, and the global flag start is set to `TRUE`. From this point on, each worker thread evolves through its own sub-state machine, and the web server remains available to any browser that connects.

Within Polling Active, each worker thread goes through the following sub-states:

Connecting — the thread creates a `CSocket` and attempts to connect to its slave. This is a brief transient state.

Polling loop — once the connection succeeds, the thread enters a loop that builds the appropriate Modbus request (FC 0x03 for Motor and Accionamientos, FC 0x06 for Luces), sends it, parses the response, updates the shared state of the dialog and sleeps for the configured interval before repeating.

Terminated — the thread leaves the loop for one of two reasons: the user has pressed Stop (which sets start to FALSE) or the connection has been lost (a call to Send or Receive returned a non-positive value). The thread closes its socket, sets its connection LED to OFF and exits. From that point on, that particular slave is unavailable until the user presses Stop and Start again.

Pressing Stop transitions the application back to Idle. The flag start becomes FALSE, the worker threads detect the change at the next iteration of their loop and exit cleanly, and the web server's listening socket is closed.

The diagram below (Figure 4) summarises this behaviour. The outer machine represents the global life cycle of the application; the inner machine, contained inside Polling Active, describes what each worker thread does independently.

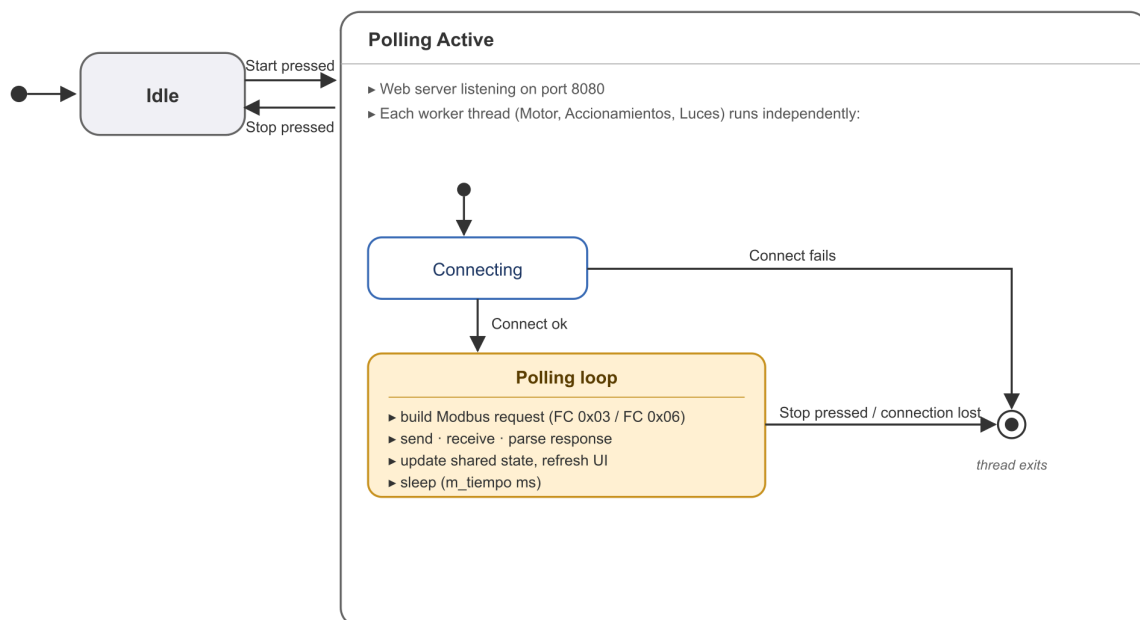


Figure 4 – State diagram of the Master

3. Flowcharts of the Main Functions

This section presents the algorithmic flow of the most relevant operations executed by the Centralita: the polling logic of a reader worker thread, the polling logic of the writer worker thread, and the HTTP request handler of the embedded web server. Together they cover the three behavioural patterns that the application repeats throughout its life cycle.

Thread for Slave 1: Motor

The Motor function is launched in its own thread by `AfxBeginThread` when the user presses Start. It connects to the Motor slave, then iterates a fixed read-update cycle until either the user requests a stop or the connection breaks. Figure 5 shows the whole process.

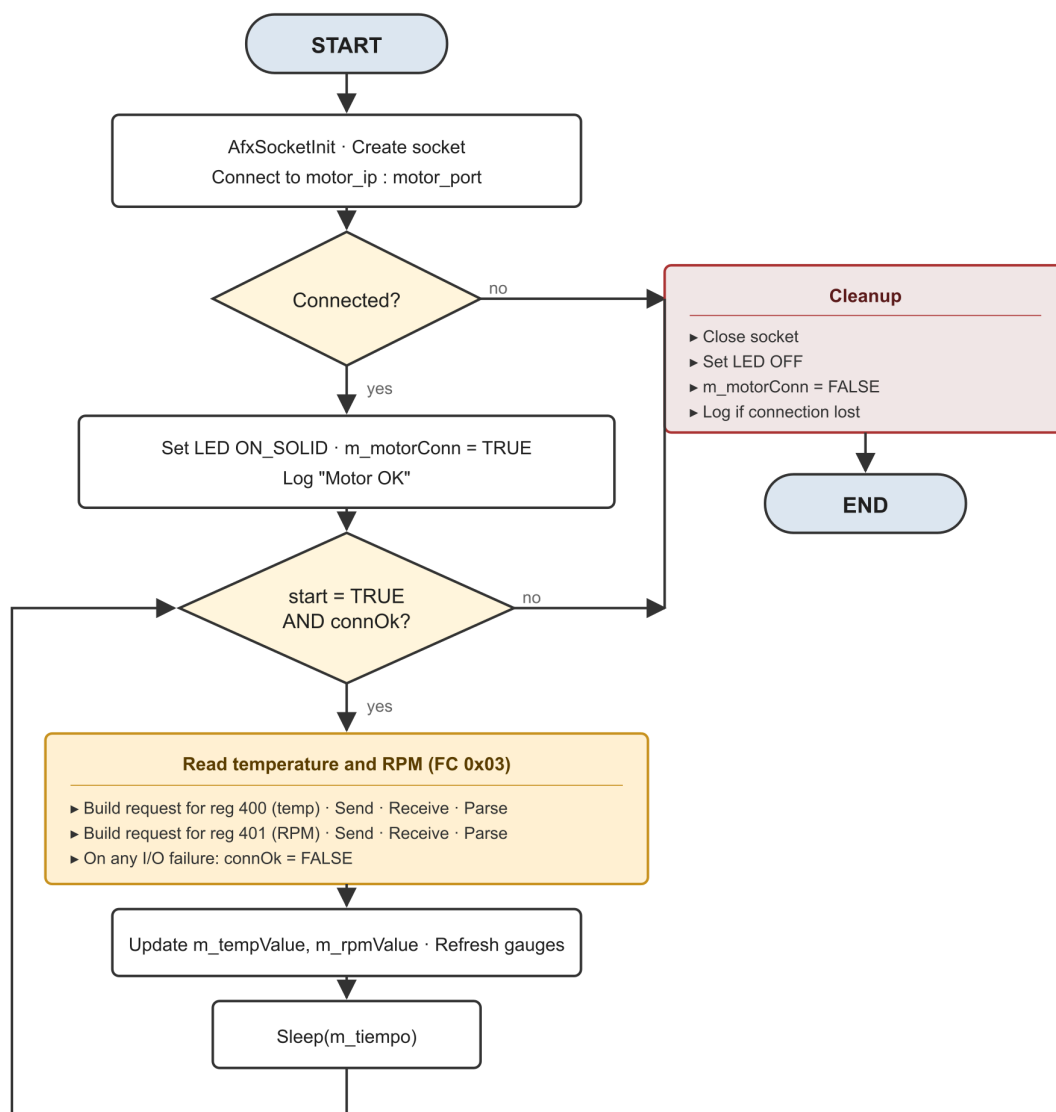


Figure 5 – Flowchart for the thread for Slave 1: Motor

Thread for Slave 2: Accionamientos

The thread for Slave 2: Accionamientos follows exactly the same structure, with the only differences that it issues three FC 0x03 read requests (registers 400, 401 and 402) per cycle instead of two, and that the values it stores in the shared state (`m_frenoState`, `m_izqState`, `m_derState`) are read in turn by the Luces worker to drive the lights slave. For brevity its flowchart is omitted; it is obtained by replacing the two reads in the diagram above with three.

Thread for Slave 3: Luces

The Luces thread mirrors the overall structure of Motor but differs in two significant ways: it writes instead of reading, and each iteration of its loop produces five Modbus

requests instead of two. At the start of every cycle it takes a snapshot of the actuator state that the Accionamientos worker has just published into the shared variables `m_frenoState`, `m_izqState` and `m_derState`. It then forwards that snapshot to the lights slave with five FC 0x06 Write Single Register requests — one per physical indicator on the vehicle: register 500 for the brake light, registers 501 and 502 for the front left and right turn signals, and registers 503 and 504 for the rear left and right turn signals. The slave acknowledges every write by echoing the request back; Luces parses the echo only to consume the bytes from the socket, since the master already knows the value it has just sent. Error handling follows the same convention as the reader workers: a failed Send or Receive clears `connOk` and the loop exits through the same cleanup block, ensuring that the socket is closed and the connection state is reset. Figure 6 shows the whole process.

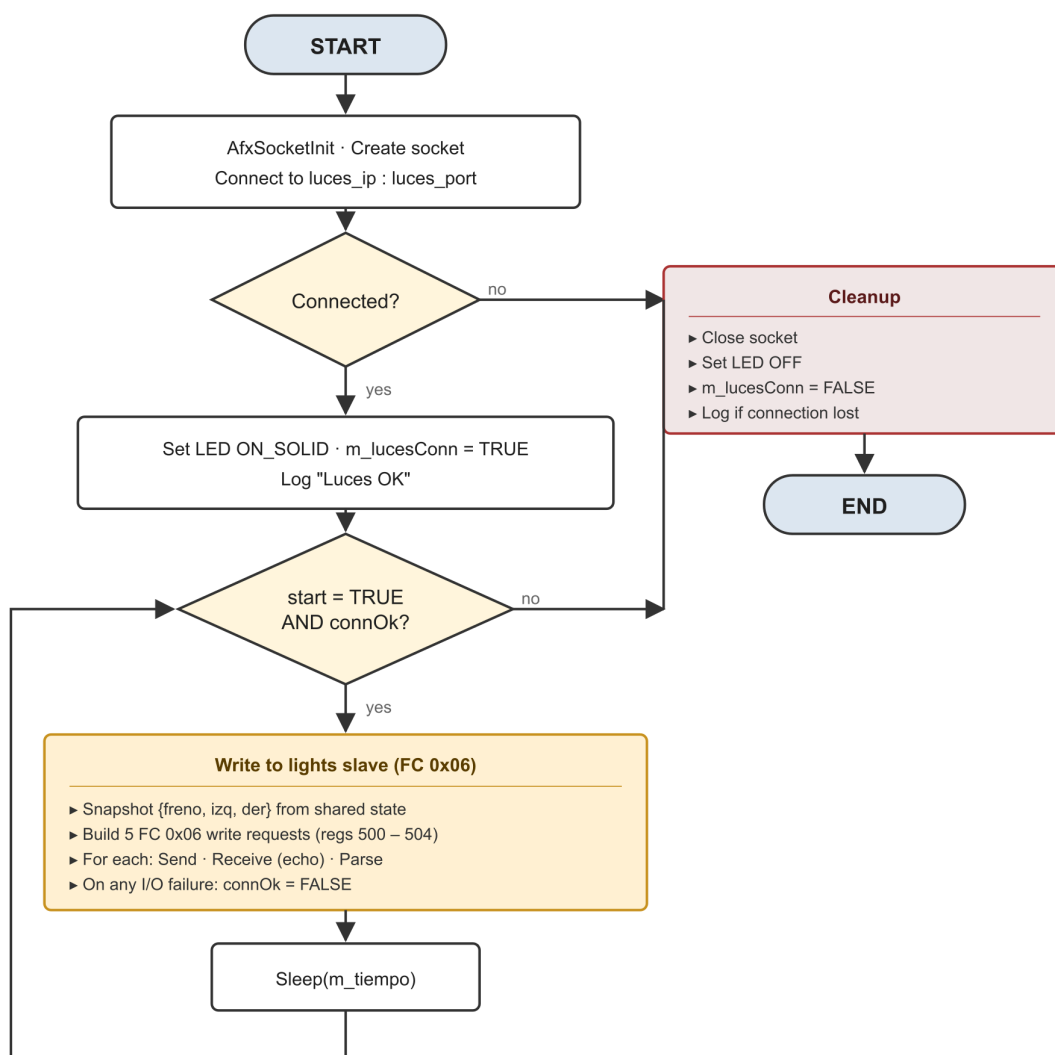


Figure 6 – Flowchart for the thread for Slave 3: Luces

HTTP request handler — OnWebAccept

`OnWebAccept` is the method called by `CWebSocket::OnAccept` whenever a browser opens a connection to the embedded web server. Unlike the worker threads, it is invoked

from the dialog's main thread and runs to completion in a single pass — there is no loop. Its job is to read one HTTP request, decide which resource to serve, build the corresponding response and close the connection.

After accepting the incoming connection and receiving the request bytes from the browser, the function parses the URL by locating the substring "GET" and the trailing "HTTP" in the buffer, and extracting whatever sits between them. The resulting path determines what is sent back:

/data returns a small JSON object built by `getDataJSON`, containing the current state of the vehicle (engine values, connection flags, switch states). This is the endpoint that the JavaScript on the dashboard page polls every 300 ms.

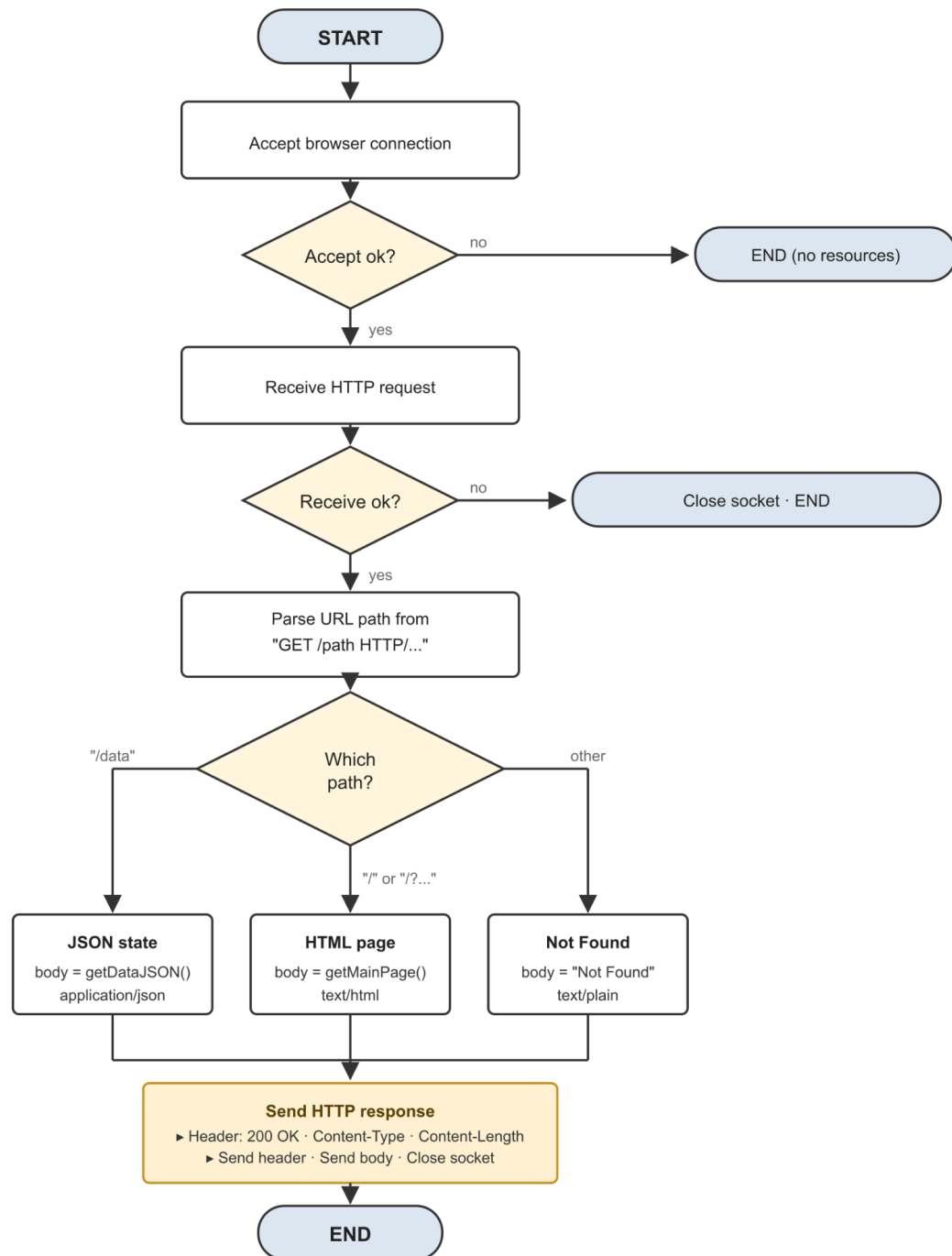


Figure 7 – Flowchart for the thread for OnWebAccept

SLAVE 1: MOTOR

This section describes the classes, functions and states of the Slave 1: Motor, which simulates two engine sensors, temperature (0–300 °C) and RPM (0–7000), driven by two sliders. Figure 8 shows the UI of the application.

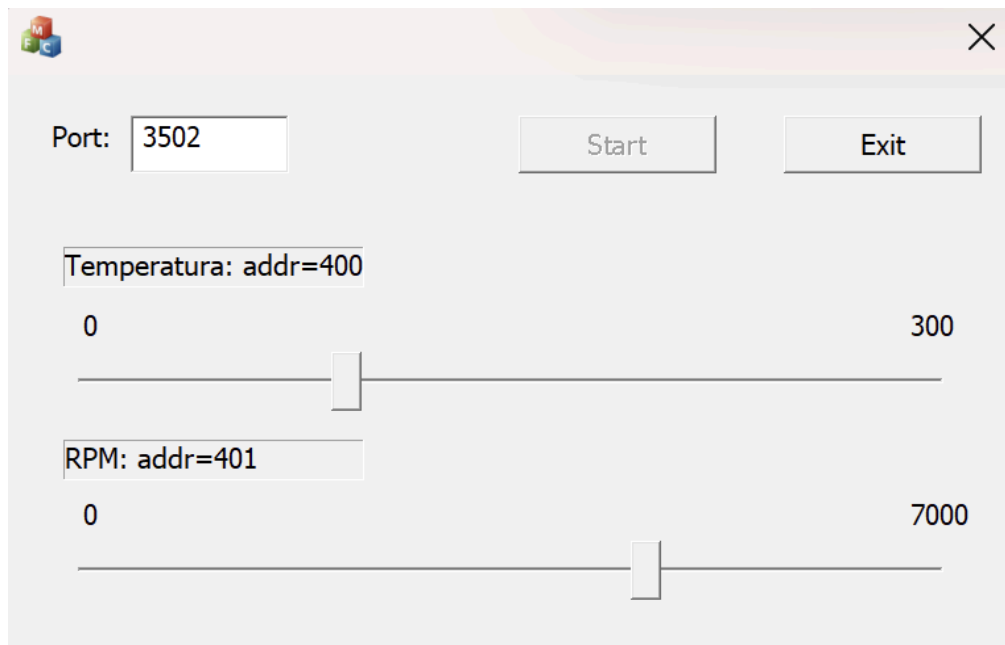


Figure 8 – Slave 1: Motor

1. Classes

The Motor slave is built around three classes. Two of them derive from the MFC framework, and the third is the same Modbus utility class already introduced in the description of the Centralita.

CEscavoMotorDlg. Derived from CDialogEx.

This is the main dialog class of the slave. It owns the user interface — two horizontal sliders that simulate the temperature and RPM sensors, plus the port configuration field — and the listening socket of the server. The current values of the sliders are stored in the public fields `m_temp` and `m_rpm`, which the worker threads read directly whenever they need to answer a Modbus read request. Its most relevant methods are:

- **OnInitDialog()** — configures the two sliders with their value ranges (0–300 for temperature, 0–7000 for RPM).
- **OnBnClickedStart()** — creates a `MotorSocket` instance on the configured port, puts it in Listen mode, and disables the Start button to prevent a second invocation.
- **OnHScroll(...)** — called whenever the user moves either slider; it refreshes `m_temp` and `m_rpm` from the controls via `UpdateData`.
- **OnBnClickedExit()** — closes the dialog.

MotorSocket. Derived from CSocket.

The listening socket of the slave server. Its only responsibility is to detect incoming connections from the Centralita and dispatch each new client to its own worker thread. It overrides a single virtual method, **OnAccept**, which MFC calls automatically every time the master opens a TCP connection.

Inside **OnAccept** the listening socket accepts the incoming connection into a temporary **CSocket**, immediately detaches the underlying handle, packs it together with a pointer to the dialog into a small parameter structure, and launches the free function **AtenderCliente** with those parameters in a new worker thread via **AfxBeginThread**. From that point on the listening socket goes back to waiting for the next connection, and the actual Modbus traffic is handled by the worker thread. This workflow allows the user to keep interacting with the UI while the application is communicating with the master simultaneously.

Modbus.

The same utility class was introduced for the Centralita. The methods exercised on the slave side are:

- **ParseRequest(buf, len)** — decodes the bytes received from the master, extracting the function code, the address and, depending on the function code, either the quantity (for FC 0x03) or the value (for FC 0x06).
- **BuildReadResponse(buf, vals, count)** — assembles an FC 0x03 response with the requested register values, ready to be sent back to the master.

The class diagram below (Figure 9) summarises these relationships.

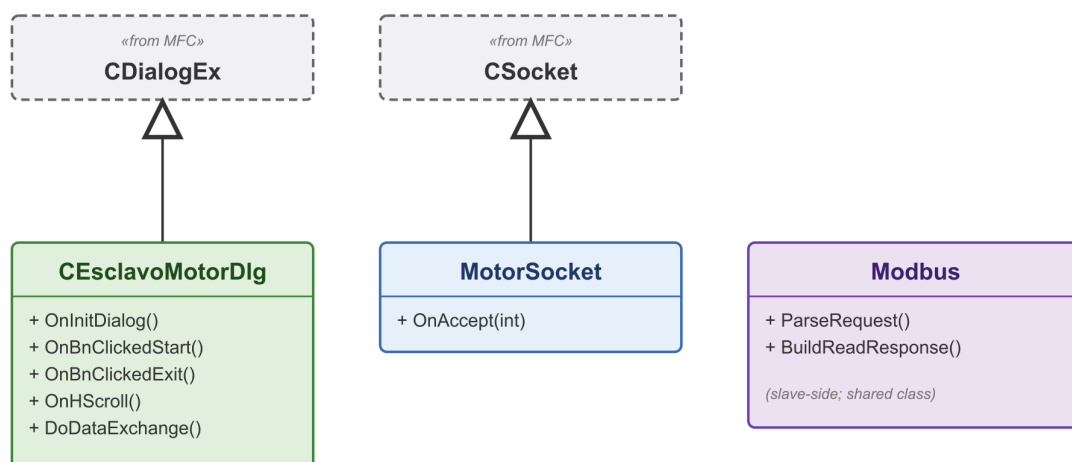


Figure 9 – Classes used in the Slave 1: Motor

2. States of the Slave 1: Motor

The state machine of the Motor slave is considerably simpler than that of the Centralita. There is no Stop-and-restart cycle and no per-slave failure handling at the dialog level: once Start has been pressed, the application stays serving requests until the user closes it.

When the application is launched it sits in the Idle state. The dialog is open and the sliders are at zero, but the listening socket has not yet been created. The user can adjust the port number in this state.

Pressing Start transitions the application to the Listening state. A MotorSocket instance is created, bound to the configured port and put into Listen mode. From that point on the slave is online: it accepts any number of incoming connections from the Centralita and stays in this state until the application is closed.

Inside the Listening state, every accepted connection causes a new worker thread (AtenderCliente) to be spawned. Each worker has its own short life cycle:

- **Service loop** — the worker waits for an incoming Modbus request on its socket. When a request arrives, it is decoded with ParseRequest, the current values of m_temp and m_rpm are read from the dialog, and an FC 0x03 read response is assembled with BuildReadResponse and sent back to the master. The worker then loops back to waiting for the next request.
- **Exit** — when the client closes the connection (typically because the Centralita's worker thread has shut down) or any Send/Receive returns a non-positive value, the loop terminates and the worker thread exits. The listening socket is not affected; it continues accepting new connections.

The diagram below (Figure 10) summarises this behaviour. The single outer transition (Start pressed) is the only state change at the application level; the inner sub-machine is replicated independently for every connected client.

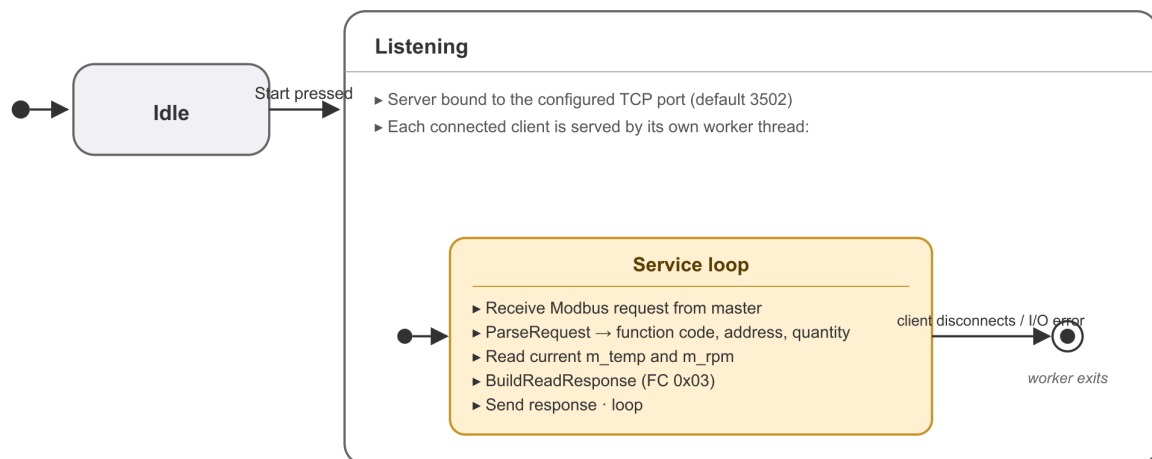


Figure 10 – State diagram of the Slave 1: Motor

3. Flowchart of the Slave 1: Motor

The behaviour of the Motor slave can be summarised in a single flowchart that walks through the three phases of its life: the setup of the listening socket on the main UI thread, the per-connection handler in `MotorSocket::OnAccept`, and the receive-and-respond loop executed by `AtenderCliente` in its own worker thread per client.

When the application is launched, the dialog is shown and stays idle until the user presses the Start button. At that moment a `MotorSocket` is created, the port is bound and the socket is put into Listen mode, and the Start button is disabled to prevent a second invocation. From that point on the application is in a continuous listening loop: every time a client opens a TCP connection, MFC delivers an `OnAccept` callback to the listening socket. The handler accepts the incoming connection, detaches the underlying socket handle, packs it together with a pointer to the dialog into a small parameter structure, and launches a new worker thread (`AtenderCliente`) with that structure as argument. The listening socket immediately returns to waiting for the next client.

In its own thread, the worker attaches the inherited socket handle and enters a loop in which it receives one Modbus request, parses it, looks up the current values of `m_temp` and `m_rpm`, builds an FC 0x03 response and sends it back to the master. The loop continues until the client closes the connection or any `Send/Receive` returns a non-positive value, at which point the worker closes its socket and the thread exits. The fork bar in the diagram (Figure 11) marks the moment at which the main UI thread and the worker thread run concurrently.

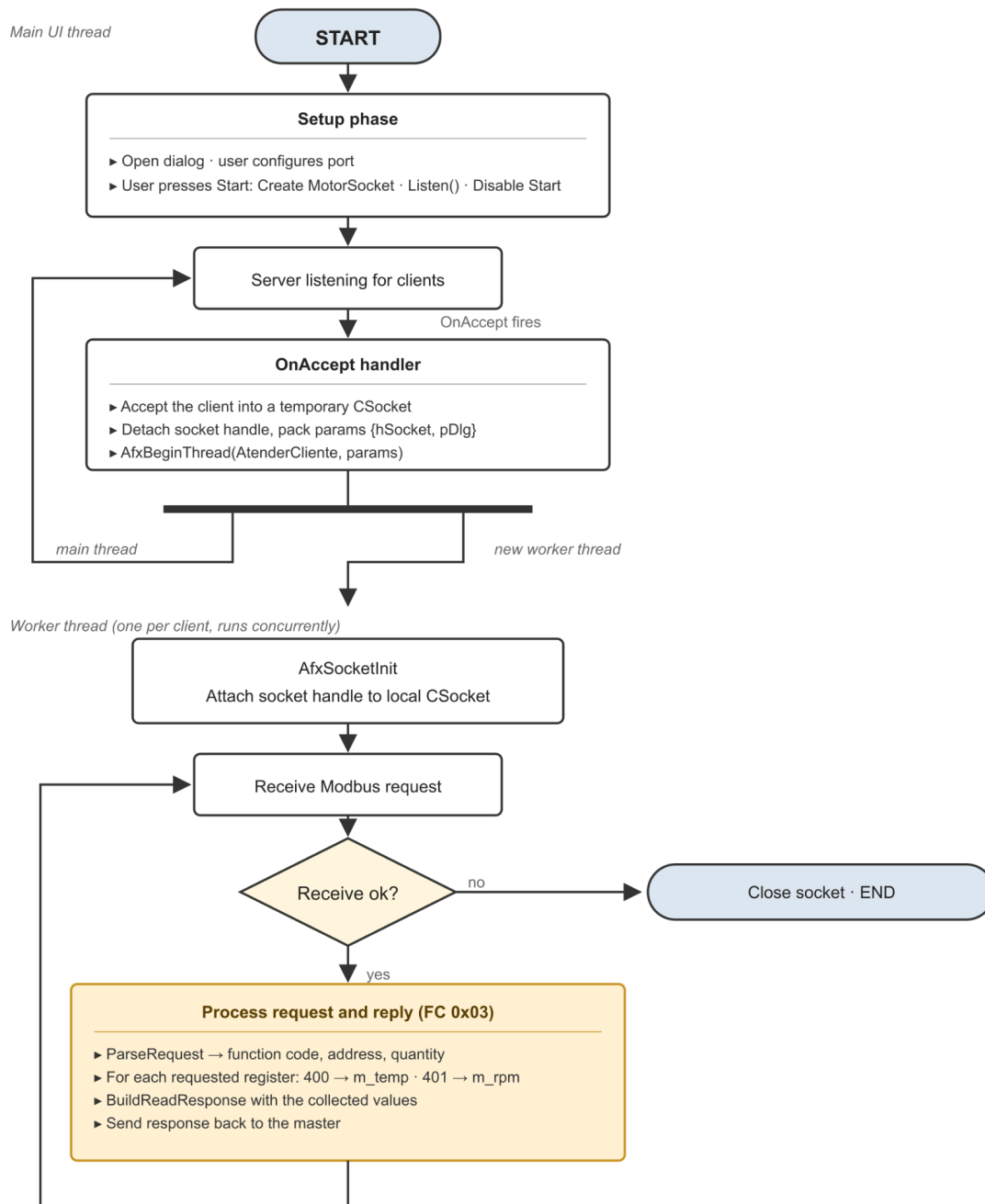


Figure 11 – Flowchart of the Slave 1: Motor

SLAVE 2: ACCIONAMIENTOS

This section describes the classes, functions and states of the Slave 1: Accionamientos, which simulates the driver inputs (brake, left turn signal and right turn signal) implemented as three check buttons. Figure 12 shows the UI of the application.

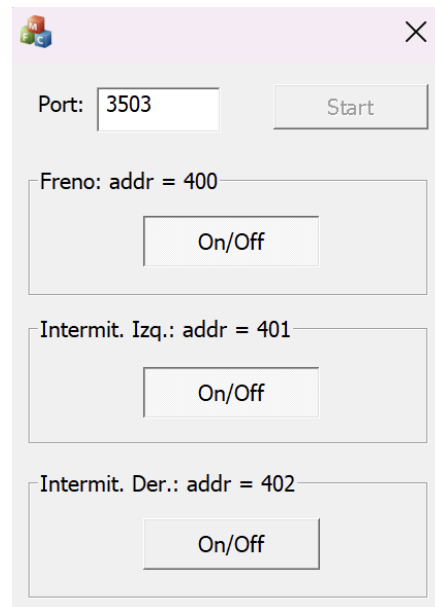


Figure 12 – Slave 2: Accionamientos

1. Classes

The Accionamientos slave is built around three classes — the same architecture used by the Motor slave. Two of them derive from the MFC framework, and the third is the same Modbus utility class introduced in the description of the Centralita.

CEsclavoAccionamientosDlg. Derived from CDialogEx.

This is the main dialog class of the slave. It owns the user interface — three check boxes that represent the brake (freno), the left turn signal (izquierdo) and the right turn signal (derecho), plus the port configuration field — and the listening socket of the server. The current states of the three switches are stored in the public BOOL fields freno, izquierdo and derecho, which the worker threads read directly whenever they need to answer a Modbus read request. Its most relevant methods are:

- **OnInitDialog()** — standard dialog initialisation (icon, system menu, About box entry); no extra setup is needed because the check boxes are already wired through DDX.
- **OnBnClickedStart()** — creates an AccionamientoSocket instance on the configured port, puts it in Listen mode and disables the Start button to prevent a second invocation.

- **OnBnClickedFreno(), OnBnClickedIzq(), OnBnClickedDer()** — message handlers for the three check boxes. Each calls `UpdateData(TRUE)` to refresh the public BOOLS from the controls so that the worker threads see the new switch state on the next request.

AccionamientoSocket. Derived from CSocket.

The listening socket of the slave server. Its responsibility is the same as that of `MotorSocket` in the previous section: detect incoming connections from the Centralita and dispatch each new client to its own worker thread. It overrides the single virtual method **OnAccept**, which MFC calls automatically whenever the master opens a TCP connection.

Inside `OnAccept` the listening socket accepts the incoming connection into a temporary `CSocket`, immediately detaches the underlying handle, packs it together with a pointer to the dialog into a small parameter structure, and launches the free function `AtenderClienteAcc` with those parameters in a new worker thread via `AfxBeginThread`. From that point on the listening socket goes back to waiting for the next connection, and the actual Modbus traffic is handled by the worker thread.

Modbus.

The same utility class introduced for the Centralita. As in the Motor slave, the methods exercised on the slave side are:

- **ParseRequest(buf, len)** — decodes the bytes received from the master.
- **BuildReadResponse(buf, vals, count)** — assembles an FC 0x03 response with the requested register values.

The class diagram below (Figure 13) summarises these relationships.

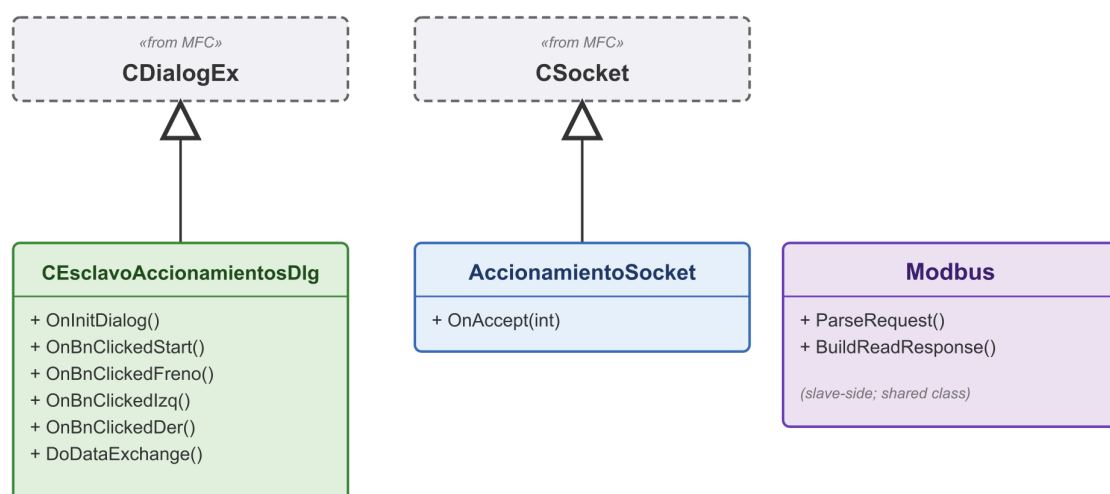


Figure 13 – Classes used in the Slave 2: Accionamientos

2. States of the Slave 2: Accionamientos

The state machine of this slave is identical in structure to the one described for the Motor slave: an outer transition from Idle to Listening triggered by the Start button, and, inside Listening, a per-worker sub-machine with a single Service loop state that the worker leaves only when the client closes the connection or an I/O error occurs.

The only differences relative to the Motor case are the default TCP port (3503 instead of 3502) and the content of the service loop, which now reads the three brake/turn-signal switches (freno, izquierdo, derecho) and answers with three register values instead of two.

The diagram (Figure 14) is included below for completeness.

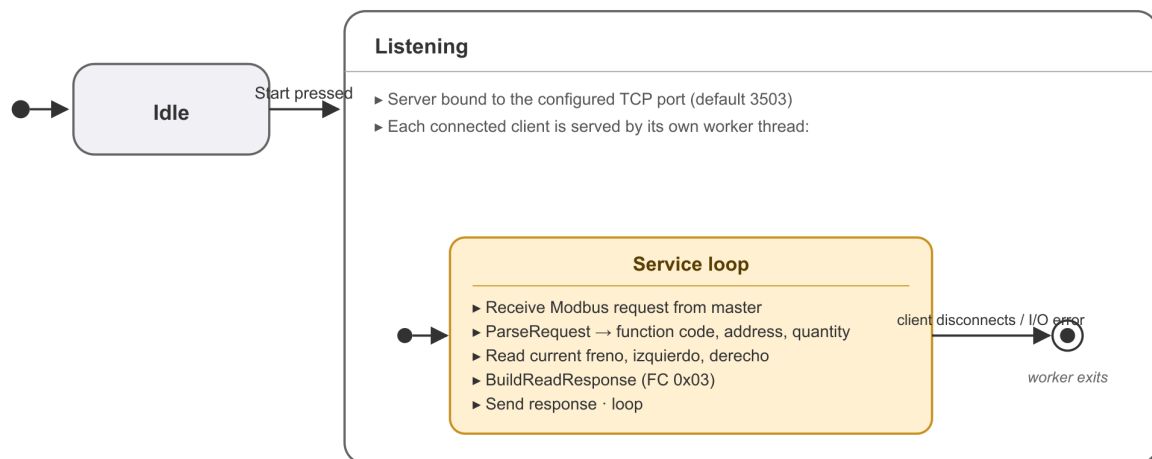


Figure 14 – State diagram of the Slave 2: Accionamientos

3. Flowchart of the Slave 2: Accionamientos

The behaviour of the Accionamientos slave follows exactly the same three-phase structure as the Motor slave: setup on the main UI thread, an OnAccept handler that spawns a worker thread per client, and a worker thread executing a receive-and-respond loop. The differences are limited to the names of the classes and worker function involved (AccionamientoSocket and AtenderClienteAcc), and to the registers read inside the loop, which now correspond to the three switches freno (register 400), izquierdo (register 401) and derecho (register 402). Each switch is encoded as a single Modbus register value — 0 if the check box is unchecked, 1 if checked — before the response is built.

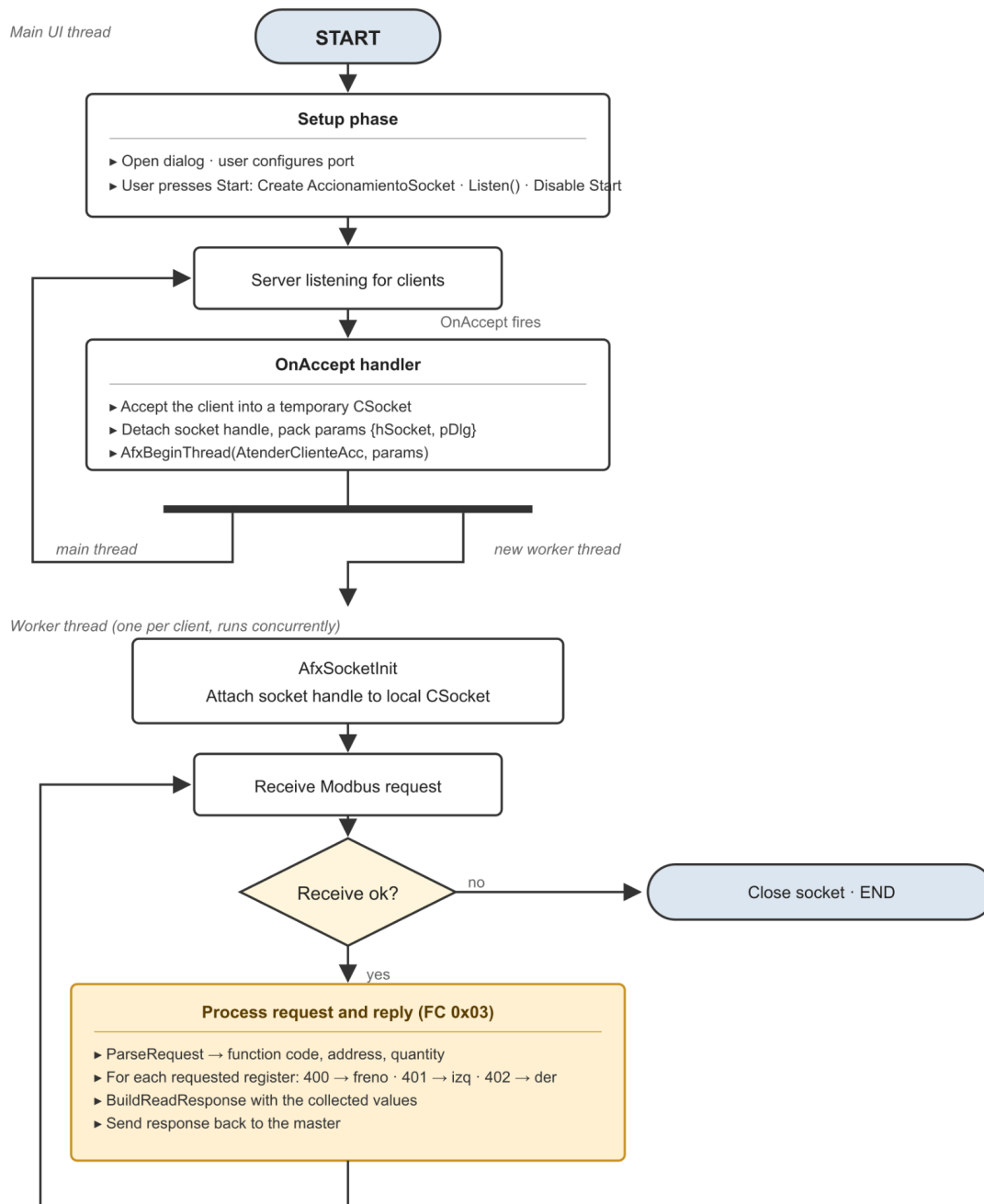


Figure 15 – Flowchart of the Slave 2: Accionamientos

SLAVE 3: LUCES

This section describes the classes, functions and states of the Slave 3: Luces, which simulates the physical lights of the vehicle: brake light, two front turn signals and two rear turn signals. Figure 16 shows the UI of the application.

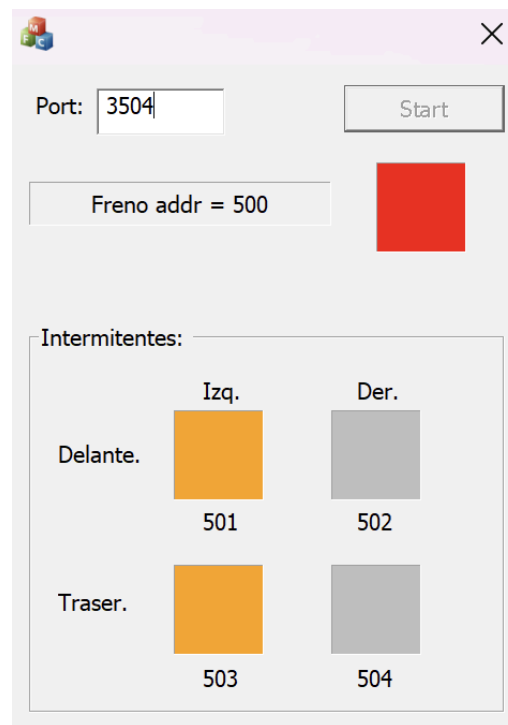


Figure 16 – Slave 3: Luces

1. Classes

The Luces slave is built around four classes, one more than the previous two slaves. Three of them derive from the MFC framework, and the fourth is the same Modbus utility class introduced in the description of the Centralita. The extra class compared to Motor and Accionamientos is Led, the custom indicator control that was first described in the Master section — Luces is the only slave that needs visual indicators on its dialog.

CEsclavoLucesDlg. Derived from CDialogEx.

This is the main dialog class of the slave. It owns the user interface — five Led controls representing the brake light (led_freno), the two front turn signals (led_izq_del, led_der_del) and the two rear turn signals (led_izq_tras, led_der_tras), plus the port configuration field — and the listening socket of the server. Unlike the previous slaves, the dialog also drives a timer that animates the blinking of the turn-signal LEDs. Its most relevant methods are:

- **OnInitDialog()** — configures the five LEDs with their colors (red for the brake light, amber for the four turn signals) and starts the dialog timer with a 400 ms period.
- **OnBnClickedStart()** — creates a `LucesSocket` instance on the configured port, puts it in Listen mode and disables the Start button.
- **OnTimer()** — fires every 400 ms and calls `Tick()` on the four turn-signal LEDs to advance their blink animation. The brake LED is not ticked because it operates in solid mode.
- **DoDataExchange()** — wires the port field and the five LED controls to the dialog via DDX.

LucesSocket. Derived from CSocket.

The listening socket of the slave server. Same role and same pattern as `MotorSocket` and `AccionamientoSocket` in the previous sections: it overrides **OnAccept**, which MFC calls every time the master opens a TCP connection, and uses that callback to accept the client, detach the underlying handle and spawn a new worker thread by calling `AfxBeginThread` on the free function `AtenderClienteLuces`.

Led. Derived from CStatic.

The same custom control already described for the `Centralita`; it is reused unchanged in this slave to render the five vehicle lights. Its public interface (`SetColor`, `SetMode`, `Tick`, `OnPaint`) is documented in that section and is not repeated here.

Modbus.

The same utility class. The slave-side methods exercised here are different from those used in the two previous slaves, because `Luces` handles writes rather than reads:

- **ParseRequest(buf, len)** — decodes the bytes received from the master. For FC 0x06 it extracts the register address and the value to write.
- **BuildWriteResponseEcho(buf)** — builds the FC 0x06 acknowledgement, which by Modbus convention is a byte-for-byte echo of the request that the slave just received.

The class diagram (Figure 17) below summarises these relationships.

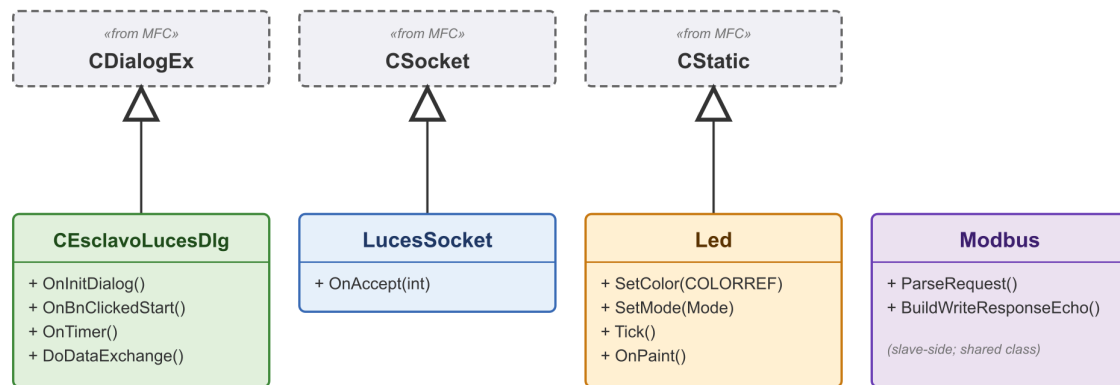


Figure 17 – Classes used in the Slave 3: Luces

2. States of the Slave 3: Luces

The top-level state machine is identical to that of the Motor and Accionamientos slaves: Idle → Listening, with a per-worker sub-machine inside Listening. The differences relative to previous slaves are the default port (3504), the content of the Service loop (which now handles writes instead of reads), and the existence of a dialog timer that runs in parallel to drive the LED blink animation.

When a worker thread receives a request, it parses the function code (FC 0x06) and the target register address, updates the corresponding LED on the dialog — register 500 controls the brake light in ON_SOLID mode, registers 501–504 control the four turn signals in ON_BLINKING mode — and replies to the master with a byte-for-byte echo of the request, which is the standard Modbus acknowledgement for FC 0x06.

Meanwhile, a separate dialog timer set up at startup fires every 400 ms and calls Tick() on each turn-signal LED. The timer is unrelated to the Modbus traffic — it only advances the blink animation of those LEDs that are currently in ON_BLINKING mode. This means the visual blinking is driven entirely by the dialog timer, while the on/off state of each LED is determined exclusively by the latest FC 0x06 write received from the master.

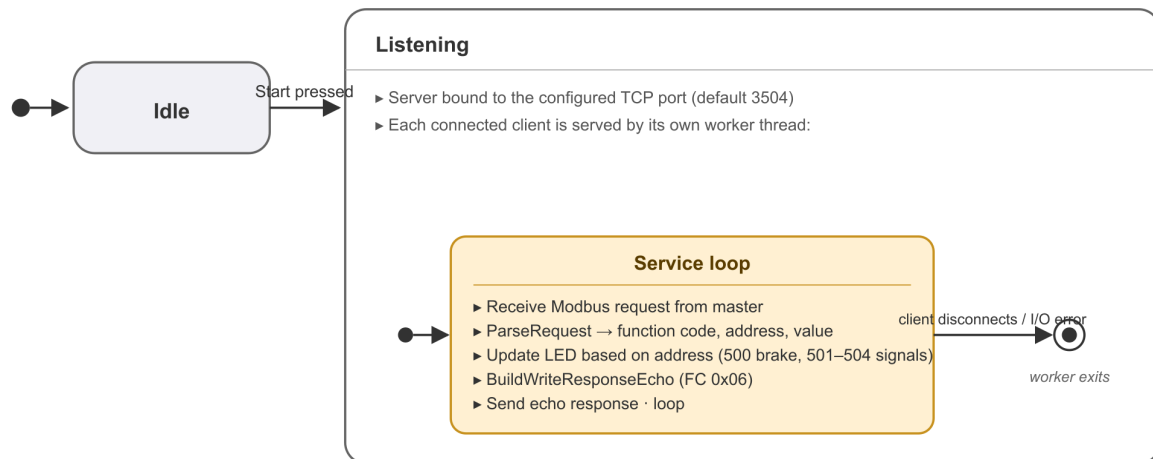


Figure 18 – State diagram of the Slave 3: Lucex

3. Flowchart of the Slave 3: Lucex

The behaviour of the Lucex slave follows the same three-phase structure as the previous slaves: setup on the main UI thread, an OnAccept handler that spawns a worker thread per client, and a worker thread executing a receive-and-respond loop. The differences relative to previous slaves are in the names of the classes and worker function (LucexSocket, AtenderClienteLucex), in the function code handled inside the loop (FC 0x06 instead of FC 0x03) and, most importantly, in what each request does: rather than reading a register, the worker updates the on-screen LED that corresponds to the target address.

The address-to-LED mapping inside the worker is a small switch statement. Register 500 maps to the brake light and is set to ON_SOLID; registers 501–504 map to the four turn signals and are set to ON_BLINKING. In all five cases, a value of 1 turns the LED on in the appropriate mode and a value of 0 turns it off. The response to every write is a byte-for-byte echo of the request, built with BuildWriteResponseEcho.

The flowchart (Figure 19) does not show the dialog timer that animates the blinking. As discussed in section 2, that timer runs in parallel on the UI thread and is independent of the Modbus flow.

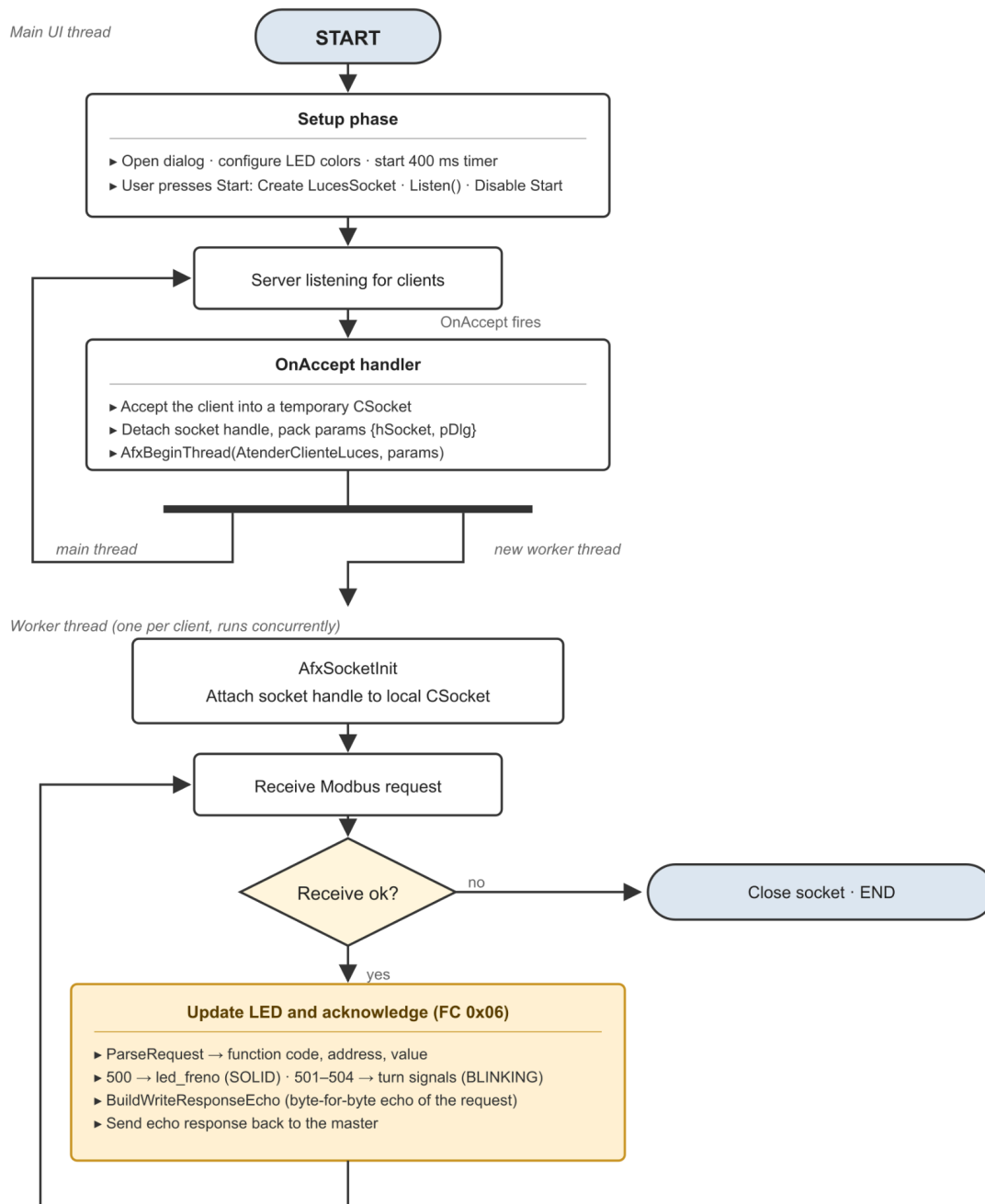


Figure 19 – Flowchart of Slave 3: Luces

WEB SERVER

The Centralita includes a small embedded HTTP server that exposes the current state of the vehicle through a web page. Any browser running on the same machine can open the page and see the same information that the dialog displays: engine temperature, RPM, brake light, turn signals, and the connection status of each slave, updated several times per second.

The server is not a separate application. It lives inside the main dialog of the Centralita and reuses the same state variables that the worker threads write into. Its listening socket (`s_listen`) is an instance of `CWebSocket`, which derives from `CAsyncSocket`. As explained in the master's section, the choice of `CAsyncSocket` is what makes this possible: incoming TCP connections are signalled to the application through the asynchronous `OnAccept` callback, so the same thread that handles user input also handles HTTP requests as they arrive, without blocking the dialog's message loop. The flowchart of `OnWebAccept` in the master's section describes the routing in detail; this section adds the context of the underlying protocol.

How the HTTP exchange works. HTTP is a stateless request/response protocol that runs on top of TCP. Each interaction with the server consists of one TCP connection in which the browser sends a request, the server sends a response, and the connection is closed. There are no sessions, no persistent state between requests, and (in the version used by this project) no compression, encryption or authentication.

The HTML returned by `getMainPage` is a single self-contained document with embedded CSS and JavaScript. The HTML defines the visual structure (gauges for temperature and RPM, indicator boxes for the lights, connection-status badges), the CSS gives it a dark dashboard-like appearance, and the JavaScript is responsible for keeping the page up to date. The update mechanism is a textbook AJAX polling pattern: every 300 milliseconds, the page calls `fetch('/data')` to ask the server for the latest state, parses the JSON response, and updates the DOM with the new values. The page is never reloaded — only the parts that change are touched — which produces the impression of a live dashboard without any complex client-server protocol. The dashboard can be seen in Figure 20.

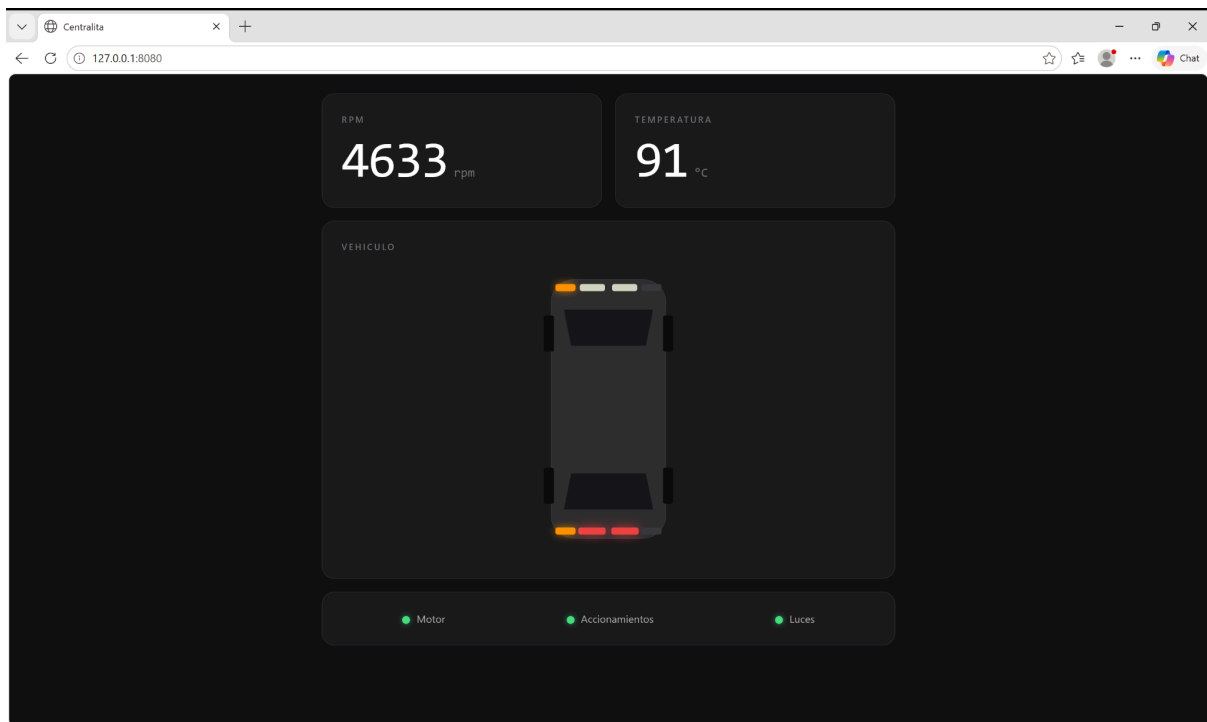


Figure 20 – Web server

CONCLUSION

This project has resulted in a distributed control system that emulates the electronic architecture of a car, built around four independent MFC applications that communicate over Modbus TCP. The Centralita acts as the master and coordinates three slave devices: the engine (temperature and RPM sensors), the actuators (brake and turn signals) and the lights (front and rear indicators, brake light). The communication uses two Modbus function codes — FC 0x03 (Read Holding Registers) to poll the engine and actuator slaves, and FC 0x06 (Write Single Register) to drive the lights — and follows the standard request/response pattern of the protocol over TCP/IP.

The Centralita parallelises its polling by running three worker threads, one per slave, so that the user interface remains responsive and the three subsystems are queried simultaneously. The same application also embeds a small HTTP server that exposes the current state of the vehicle through a web dashboard. The dashboard, written in HTML, CSS and JavaScript and served by the Centralita itself, polls a JSON endpoint several times per second and reflects the engine values, the actuator switches and the connection status of each slave in real time, without ever reloading the page.

The final result is a working master–slave system that brings together the three main technical topics covered during the course: the Modbus protocol over TCP/IP, the use of concurrency with MFC worker threads, and the integration of an embedded web server into a desktop application.